POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Project Report

Customize RISC-V ISA for AI Kernel Acceleration: the `mac_32` and `mac_8` Instructions

Advanced Computer Architectures

Author: Giuliano Crescimbeni

Advisor: Prof. Cristina Silvano

Co-advisor: Ing. Tommaso Spagnolo

Academic year: 2025-2026

1. Introduction

Machine-learning inference and digital signal processing are increasingly pushed onto small, in-order RISC-V cores [1], where tight energy and area budgets rule out wide out-of-order machines. The kernels that dominate these workloads, the matrix multiplications and convolutions at the heart of neural-network inference, reduce, at instruction level, to an overwhelming stream of *multiply–accumulate* (MAC) operations (A long running sum $acc \leftarrow acc + a \cdot b$). On a stock RV32IM core every MAC costs a separate `mul` and `add`, and on a pipelined multiplier the `add` additionally stalls on the product, the recurring `mul; add` pair is therefore the natural target for instruction-set acceleration.

This work targets *Tiny-Vedas* [2], a compact 4-stage in-order RV32IM core (IFU → IDU0 → IDU1 → EXU) with register forwarding, control-hazard flush and multi-cycle multiplier and divider units, and accelerates its MAC-bound inner loops with two custom instructions — `mac_32` and `mac_8` — implemented for this project specifically to accelerate the recurring `mul; add` blocks identified above. `mac_32` is a scalar fused multiply–accumulate, $rd \leftarrow rd + rs1 \cdot rs2$ at full 32-bit precision, that collapses the `mul; add` pair into a single operation. `mac_8` is a sub-word SIMD instruction that performs four signed INT8 multiply–accumulates per instruction, trading numerical precision for arithmetic density.

Because a 32-bit operand and four packed INT8 operands carry the same number of bits, the two units are compared at an equal *operand-bit* budget — the MAC-per-bit criterion — which exposes the performance/precision trade-off rather than hiding it. The contributions are: the RTL design and integration of both units inside the existing pipeline and a set of testbench for performance evaluation, each runned on three configurations — `standard` (RV32IM `mul+add`), `mac_32` and `mac_8`.

2. Implementation

The baseline Tiny-Vedas core is a 4-stage in-order RV32IM pipeline (Figure 1): instruction fetch (IF/IFU), a two-step decode front end (ID0/IDU0 reads the register file, ID1/IDU1 dispatches and resolves hazards), and execute (EX/EXU), where the functional units sit and results are written back. Both custom units are added inside the EXU and reuse the existing register-read and forwarding paths.

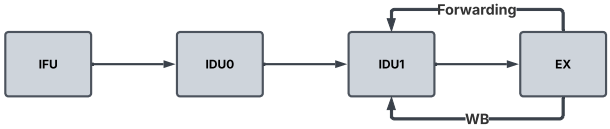

Figure 1: The 4-stage in-order Tiny-Vedas pipeline (IF → ID0 → ID1 → EX, with write-back and forwarding)

2.1. Encoding and semantics

Both instructions live in the RISC-V *Custom-0* opcode space (`opcode = 0x0B`) in the R-Type format and share the M-extension prefix `funct7 = 0000001`, differing only in the `funct3` selector (Table 1); the decoder change is minimal and no standard RV32IM encoding is touched. Both are *destructive*: the destination `rd` doubles as the third source (the accumulator), so no extra encoding bits are needed.

R-Type encoding

	funct7 [31:25]	rs2 [24:20]	rs1 [19:15]	funct3 [14:12]	rd [11:7]	opcode [6:0]
mac_32	0000001	—	—	100	—	0001011
mac_8	0000001	—	—	101	—	0001011

Table 1: R-Type encoding of the two custom instructions. They share opcode `0x0B` (Custom-0) and the M-extension `funct7` prefix, and differ only in `funct3` (`100` = `mac_32`, `101` = `mac_8`). `rd` is both destination and accumulator source.

The semantics are, for `mac_32`,

$$rd \leftarrow rd + rs1 \cdot rs2,$$

and, for `mac_8`,

$$rd \leftarrow \mathrm{sat}_{32}\left(rd + \sum_{i=0}^{3} rs1_{[8i+7:8i]} \cdot rs2_{[8i+7:8i]}\right),$$

with signed 8-bit lanes and a signed 32-bit saturating accumulation so a long sum never silently overflows. In C the instructions are emitted through the GCC directive `.insn r 0x0B,0x4,0x1,...` and `.insn r 0x0B,0x5,0x1,...`.

2.2. The mac_32 and mac_8 units

`mac_32` is grafted onto the existing 3-stage multiplier: a 32-bit adder at stage E3 folds the product into the accumulator before write-back. A dedicated third register-file read port supplies the accumulator `rs3` (aliased to `rd`), the register scoreboard (RSB) is extended to catch read-after-write hazards on `rd`, and the EXU→IDU1 forwarding is replicated for the third operand. As a result the issue cost of a `mac_32` is exactly the multiplier's 3-cycle latency, with no extra stalls introduced by the extension.

`mac_8` is instead a dedicated single-cycle combinational unit placed beside the **ALU/MUL/DIV/LSU**, shown in Figure 2. A single cycle is enough because **four parallel** 8×8 **multiplications** are far cheaper to compute than one 32×32 **multiplication**. The parallel 8×8 signed multipliers feed an 18-bit adder tree; the resulting dot product is added to the 32-bit accumulator on 33 bits (one guard bit for overflow detection) and saturated. A single output flip-flop registers the result, so write-back lands at EX+1, exactly like the ALU. The decoder marks the instruction as `mac`, reusing the same accumulator infrastructure (third read port, forwarding, RSB), but *not* as `mul`, so it never enters the multiplier pipeline nor asserts the busy/scoreboard logic and, for hazard purposes, behaves as a single-cycle operation. `mac_8` therefore attacks two axes at once: sub-word data parallelism (four lanes) and latency (one cycle instead of three).

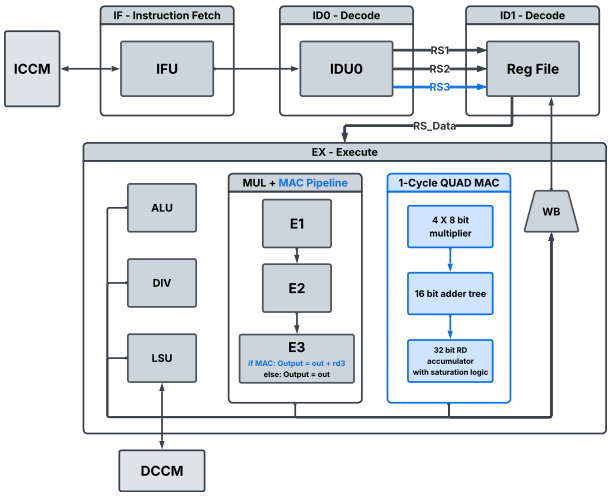

Figure 2: The two units inside the EXU: `mac_32` fused into the 3-stage multiplier, and `mac_8` as a dedicated single-cycle unit, both sharing the accumulator path (third read port and forwarding).

Because every functional unit drives one OR-combined write-back bus, dispatch must guar-

antee a single write-back per cycle. The load-dense convolution testbench (Section 3) surfaced — and led to fixing — a corner case in which a load's write-back from the third LSU stage (EX+3) could collide with a single-cycle `mac_8`/ALU write-back (EX+1); the fix holds the single-cycle producer in execute while the LSU is busy.

3. Testing Environment, Compiler and Testbenches

3.1. Verification flow

The RTL is simulated cycle-accurately with Verilator [3], and a Python RV32IM instruction-set simulator (**ISS**) acts as the golden model. Each C test is compiled twice, switched by the `USE_MAC_INSN` symbol: a portable `acc + a*b` expansion (or a scalar four-lane reference for `mac_8`) for the ISS, and the custom `.insn` build for the RTL — so the source is identical and any difference comes purely from the ISA extension. The dual-ELF flow signs off correctness by requiring both builds to leave identical final state in the committed result globals; operand magnitudes are kept small so that no overflow or saturation occurs and the two must match bit-for-bit.

3.2. Compiler

Programs are built with `riscv64-unknown-elf-gcc` for `rv32im/ilp32`. For the optimization level: at `-O0` each `mac()` call expands to about 17 instructions dominated by stack traffic, which masks the single-instruction saving; at `-O2` full inlining shrinks the loop body to roughly 8 instructions and exposes the real `mul+add`-versus-MAC trade-off. All measurements therefore use `-O2`.

3.3. Testbenches

A `mac_32` and a `mac_8` each move 64 operand bits per instruction, so a fair equal-bit comparison issues the *same number of accelerator instructions*; this makes `mac_8` perform four times the MAC arithmetic (at INT8 precision) for the same operand-bit volume. Each testbench is run on `standard`, `mac_32` and `mac_8`:

- **mac_bench** — a length-64 dot product repeated into a single register-resident accumulator, in both a C and a hand-written assembly version;
- **unrolled** — the same kernel unrolled 8× over one accumulator (loop overhead amortised, RAW chain kept);
- **par8** — 8× unrolled with eight independent accumulators (software register renaming), which breaks the accumulator chain;
- **conv2d** — a realistic "valid" 2D convolution with a 4×4 kernel.

4. Results

Tables 2 and 3 give the per-testbench instruction counts and the cycles/CPI on the three configurations under the MAC-per-bit convention, and Figure 3 (Section 4.2) visualises the cross-test comparison.

4.1. Per-testbench metrics

Retired instructions

Testbench	standard	Δ mac_32	Δ mac_8
mac_bench (C)	528 419	−65 529	−64 762
mac_bench (asm)	400 010	−100 000	−99 998
unrolled (asm)	225 010	−100 000	−99 998
par8 (asm)	225 024	−100 000	−99 998
conv2d (C)	571 223	−40 000	+633 093

Table 2: Total retired instructions for `standard`, and each accelerator's change relative to it.

Cycles, reduction and speedup

Testbench	standard	mac_32	mac_8
mac_bench (C)	1 184 921 CPI 2.24	−65 519 1.06× CPI 2.42	−195 696 1.20× CPI 2.13
mac_bench (asm)	1 200 010 CPI 3.00	−200 000 1.20× CPI 3.33	−399 998 1.50× CPI 2.67
unrolled (asm)	587 510 CPI 2.61	−112 500 1.24× CPI 3.80	−399 998 3.13× CPI 1.50
par8 (asm)	587 524 CPI 2.61	−375 000 2.76× CPI 1.70	−399 998 3.13× CPI 1.50
conv2d (C)	1 005 802 CPI 1.76	−40 000 1.04× CPI 1.82	+795 563 0.56× CPI 1.50

Table 3: For `standard`, total cycles; for the accelerators, the cycle change $\Delta = c_{\text{config}} - c_{\text{standard}}$. Below each cell, in blue the speedup $c_{\text{standard}}/c_{\text{config}}$ (all three process the same operand-bit workload) and in grey the CPI.

4.2. Cross-test comparison

Figure 3 reads as "how many times faster than `standard`": for each testbench it plots the speedup $c_{\text{standard}}/c_{\text{config}}$ (how many times fewer cycles each accelerator needs to chew through the same operand-bit workload). The two regimes are immediate: `mac_32` only takes off on `par8` (2.76×), whereas `mac_8` — four INT8 MACs per instruction — is 1.2 to 3.1× faster on the synthetic kernels and dips to 0.56× on the `conv2d`.

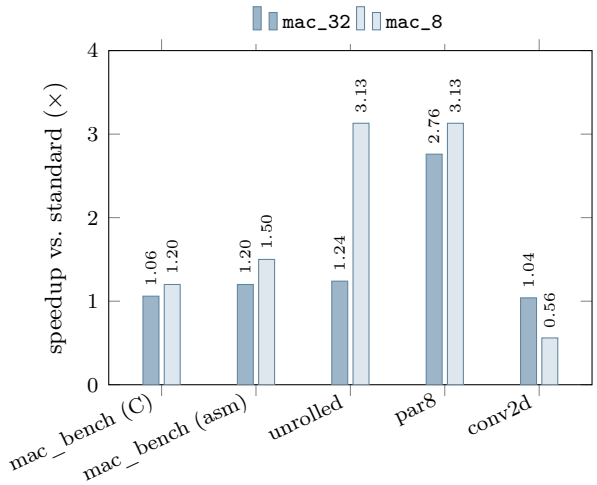

Figure 3: Speedup over `standard` (= 1×) at equal operand-bit workload, $c_{\text{standard}}/c_{\text{config}}$ (higher is faster). The `mac_8` bar already includes its four-lane INT8 parallelism.

4.3. Discussion

The data splits into two regimes. `mac_32` removes one `add` per MAC, cutting instructions and cycles against `standard`, but every MAC traverses the 3-stage multiplier, so CPI rises and, on a single accumulator, back-to-back MACs serialise on the read-after-write chain, the unrolled case reaches a CPI of 3.80. This is why loop unrolling *without* register renaming barely helps `mac_32`: every MAC reads the accumulator that the previous one writes, so the dependency chain between two consecutive MACs on the same `rd` forces a one-cycle stall on top of the 3-stage latency, roughly 4 cycles per MAC instead of 3, and merely replicating the loop body cannot overlap operations that are inherently serialised. Only *par8* (software register renaming) lets the multiplier stream independent operations, which is why `mac_32` collapses from 475 010 to 212 524 cycles there: this is the single transformation that exposes its true value.

`mac_8`, being single-cycle, never suffers the accumulator chain: *unrolled* and *par8* land within 0.1% of each other (both ≈187 500 cycles, CPI ≈ 1.50), so the renaming that `mac_32` needs is irrelevant to it. At equal operand bits it is 1.2 to 1.5× faster than `standard` on the basic kernels and 3.1× once unrolled, with CPI *improving* rather than inflating, all while computing four INT8 MACs per instruction. This speed, however, is bought with precision: `mac_8` trades the full 32-bit accuracy of `mac_32` for 8-bit lanes, so its single-cycle throughput is only usable where the application can tolerate the reduced numerical precision of INT8 arithmetic.

The realistic convolution is the counterpoint: there `mac_8` is 0.56×, *slower* than `standard` for the same operand bits, because each `mac_8` must first assemble its packed operand from four byte loads and shifts, and this marshalling, not the arithmetic, dominates the loop. `mac_8` thus pays off only when the data is already laid out as packed INT8 and the application tolerates reduced precision.

5. Conclusions

`mac_32` and `mac_8` occupy opposite points of the design space. `mac_32` fuses `mul; add` into the 3-stage multiplier: its benefit is real but conditional, bounded by the accumulator chain on single-accumulator kernels and fully exposed only once software register renaming streams independent MACs. `mac_8` attacks width and latency simultaneously as a single-cycle SIMD unit; evaluated under the MAC-per-bit criterion it is up to 3.1× faster than `standard` (and far ahead of `mac_32`) and improves CPI throughout, and it removes *in hardware* the very bottleneck `mac_32` had to dodge in software. The advantage is workload-dependent: overheads, as in the convolution, erodes it, and the INT8 precision must be acceptable for the target application.

References

[1] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, RISC-V International, 2019.

[2] spzbrnmrc, *Tiny-Vedas: a compact 4-stage in-order RV32IM core*, 2024. `https://github.com/spzbrnmrc/Tiny-Vedas` (accessed 2026-07-09).

[3] W. Snyder, "Verilator and SystemPerl," in *North American SystemC Users' Group, Design Automation Conference (DAC)*, 2004.